

LLM-as-a-Judge Toxicity Moderation with Gemma

Strict JSON, Abstention, Robustness and Uncertainty Evaluation

CSE 528 – Deep Learning II
Programming + Research Assignment

Model: *google/gemma-2-2b-it* | Backend: *Hugging Face Transformers* | *No fine-tuning*

Abstract

The following report presents a toxicity moderation system implemented with a small generative language model, *Gemma-2-2b-it*, used as an LLM-as-a-Judge rather than as a traditional discriminative classifier. The system returns a strict JSON moderation decision with label, category, confidence and short rationale, and supports DEFER (abstention) for uncertain cases. Three required decision mechanisms were implemented: direct generation (Method A), log – likelihood scoring over candidate completions (Method B) and self – consistency (Method C). The system was evaluated on a controlled local toxicity subset and then stress – tested with robustness attacks and a normalization – based mitigation. On the evaluated example subset, Method A achieved perfect performance with full coverage and valid JSON outputs, while Method C matched this predictive quality on covered examples but at much higher latency. Method B proved highly conservative and required threshold calibration, yet still remained weak as a final hard decision rule. Robustness results showed strong resistance to typos, spacing, repetition and most leetspeak, but revealed a major failure mode for benign quoted toxic language.

1. Aim and Assignment Objectives

Aim of the assignment was to design, implement and evaluate a toxicity moderation system that uses a small generative LLM as a judge and produces a structured moderation decision instead of a standard classifier score. Key objectives were to:

- (i) implement a working Gemma – based moderation pipeline on modest hardware, without fine – tuning
- (ii) enforce strict JSON outputs with validation and safe fallback behavior
- (iii) compare the three required decision mechanisms (A/B/C)
- (iv) support uncertainty – aware abstention through DEFER
- (v) evaluate robustness under input perturbations and demonstrate at least one mitigation
- (vi) report quality, safety, consistency and latency in a reproducible experimental setting

2. Theoretical Background

2.1 LLM – as – a – Judge

A generative model is prompted to act as an evaluator. Instead of learning a decision boundary through supervised fine – tuning, the model receives an instruction and produces a structured judgment. This makes prompt design, schema enforcement and uncertainty control central to system performance.

2.2 Strict JSON Output

Strict JSON turns the model output to API – like response that is machine – readable and auditable. The schema contained: label, category, confidence and short rationale. A validator checked field presence, label/category validity and confidence type/range. Invalid outputs were converted into a safe DEFER decision.

2.3 DEFER / Abstention

System abstains from making a final moderation decision because it is not sufficiently confident. This is important in safety – critical moderation because uncertain cases should not always be forced to a hard toxic/non-toxic decision. In practice, DEFER can be escalated to human review.

2.4 Decision Mechanisms

Method A (Direct Generation) asks the model to generate the final JSON response into a deterministic pass. Method B (log – likelihood scoring) scores candidate JSON completions for TOXIC / NON_TOXIC / DEFER and chooses the highest – scoring completion; makes it more classified and conservative. Method C (Self – Consistency) samples several stochastic JSON judgments, uses majority vote as the final label and vote agreement as an uncertainty estimate.

2.5 Robustness and Consistency

Robustness measures whether the moderation system remains reliable when inputs are intentionally obfuscated or corrupted, for example with typos, repeated letters, spacing attacks or leetspeak. Consistency measures whether the system gives similar answers across repeated runs or different random seeds. Robustness addresses evasion, while consistency reliability.

3. Methodology

System was implemented as a single Python script using Hugging Face Transformers and *google/gemma-2-2b-it* model. No fine – tuning or additional training was performed. The script included model loading, strict JSON prompting, JSON extraction and validation, the three decision mechanisms, evaluation utilities, a robustness suite, a normalization – based mitigation, consistency experiments and a CLI interface.

The model was prompted to produce exactly one JSON object with the schema:

```
{"label":"TOXIC|NON_TOXIC|DEFER","category":"insult|threat|hate|harassment|profanity|other|none","confidence":0.0,"short_rationale":"1-2 short sentences max"}.
```

When JSON was invalid, the system attempted to extract the first JSON object and validate it; invalid or untrusted outputs were converted to DEFER.

The evaluation used a controlled local subset stored as *sample_data.csv*. After an initial 8 – example sanity check, the dataset was expanded to 154 examples (73 toxic, 81 non – toxic). This subset included insults, threats, profanity, neutral statements, quoted toxic text, polite disagreement and mixed-context examples. For compute feasibility on local hardware, the main A/B/C comparison used random 30-example subsets.

3.1 Test Demo Method A

A demo CLI was implemented to classify an input text and return a strict JSON moderation decision. For the example input “you are stupid”, the direct generation mechanism returned a valid JSON object with a toxicity label, confidence score, short rationale and latency measurement. This verifies that the model pipeline and JSON validation work end – to – end.

```
PS C:\Users\ioann\Desktop\master\Deep_Learning_2\cse528_final> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (lily RemoteSigned) ; (& lily RemoteSigned) ; (& c:\Users\ioann\Desktop\master\Deep_Learning_2\cpython tox_judge_fixed.py --model google/gemma-2-2b-it demo --method A --text "you are stupid")arning_2\cse528_final>
Loading weights: 100% | 288/288 [00:25<00:00, 11.26it/s]
{
  "label": "TOXIC",
  "category": "insult",
  "confidence": 0.95,
  "short_rationale": "Direct insult using the word 'stupid'.",
  "valid_json": true,
  "latency_ms": 77720.51,
  "method": "direct",
  "toxic_score": 0.95
}
```

3.2 Test Demo Method B

Under the initial abstention threshold ($\tau = 0.60$), Method B deferred on all evaluated examples, resulting in zero coverage. Because the assignment explicitly requires a threshold – based DEFER policy and coverage – risk analysis, this behavior was treated as a calibration issue rather than a failure of implementation. For research purposes, an additional experiment was therefore conducted with a lower threshold ($\tau = 0.45$) in order to examine how coverage and predictive performance changed as abstention became less strict.

```
PS C:\Users\ioann\Desktop\master\Deep_Learning_2\cse528_final> python tox_judge_fixed.py --model google/gemma-2-2b-it eval --dataset sample_data.csv --method B --sample_n 30 --out_dir outputs_0_30
Loading weights: 100% | 288/288 [00:19:00:00, 14.40it/s]
[eval 0] 1/30
[eval 0] 2/30
[eval 0] 3/30
[eval 0] 4/30
[eval 0] 5/30
[eval 0] 6/30
[eval 0] 7/30
[eval 0] 8/30
[eval 0] 9/30
[eval 0] 10/30
[eval 0] 11/30
[eval 0] 12/30
[eval 0] 13/30
[eval 0] 14/30
[eval 0] 15/30
[eval 0] 16/30
[eval 0] 17/30
[eval 0] 18/30
[eval 0] 19/30
[eval 0] 20/30
[eval 0] 21/30
[eval 0] 22/30
[eval 0] 23/30
[eval 0] 24/30
[eval 0] 25/30
[eval 0] 26/30
[eval 0] 27/30
[eval 0] 28/30
[eval 0] 29/30
[eval 0] 30/30
{
  "n_examples": 30,
  "coverage": 0.0,
  "defer_rate": 1.0,
  "json_validity_rate": 1.0,
  "avg_latency_ms": 44651.111333333334,
  "auroc": 0.75,
  "auprc": 0.882275228425941,
  "precision": 0.0,
  "recall": 0.0,
  "f1": 0.0,
  "tn": 0,
  "tp": 0,
  "fn": 0,
  "fp": 0,
  "tpr": 0.0,
  "fpr": 0.0,
  "avr": 0.0
}
```

Method B produced a valid JSON decision but returned DEFER for the same input because its maximum label probability was below the abstention threshold. This demonstrates the uncertainty mechanism: rather than forcing a low – confidence decision, the system abstains when the likelihood – based confidence is insufficient.

```
(.venv) PS C:\Users\ioann\Desktop\master\Deep_Learning_2\cse528_final> python tox_judge_fixed.py --model google/gemma-2-2b-it demo --method B --text "you are stupid"
Loading weights: 100% | 288/288 [00:13:00:00, 21.01it/s]
{
  "label": "DEFER",
  "category": "other",
  "confidence": 0.434358,
  "short_rationale": "Chosen by mean completion likelihood; margin=0.150.",
  "latency_ms": 44654.03,
  "valid_json": true,
  "method": "loglikelihood",
  "scores": {
    "TOXIC": -1.831023,
    "NON_TOXIC": -1.408278,
    "DEFER": -1.843684
  },
  "label_probs": {
    "TOXIC": 0.284611,
    "NON_TOXIC": 0.434358,
    "DEFER": 0.28103
  },
  "toxic_score": 0.284611
}
```

3.3 Test Demo Method C

Method C used self – consistency by generating three independent JSON judgements and applying majority voting. For the input “you are stupid”, all samples voted TOXIC, giving an agreement score “1.00” and “TOXIC” prediction. This demonstrates that self – consistency provides an interpretable uncertainty signal through vote agreement. It also produced higher latency than the other methods because multiple generations are required for each input.

```
xt "you are stupid" \ioann\Desktop\master\Deep Learning 2\cse528_final>
Loading weights: 100% | 288/288 [00:18<00:00, 15.65it/s]
{
  "label": "TOXIC",
  "category": "other",
  "confidence": 1.0,
  "short_rationale": "Self-consistency over 3 samples; agreement=1.00.",
  "latency_ms": 237670.68,
  "valid_json": true,
  "method": "self_consistency",
  "votes": {
    "TOXIC": 3,
    "NON_TOXIC": 0,
    "DEFER": 0
  },
  "mean_sample_confidence": 0.933333,
  "toxic_score": 1.0
}
```

4. Experimental Results

All three decision mechanisms were first validated through CLI demos and then compared on the same example subset. Method B was additionally re – evaluated after threshold calibration because its default abstention policy was too conservative.

<i>Method</i>	<i>Coverage</i>	<i>DEFER</i>	<i>JSON</i>	<i>F1</i>	<i>AUROC</i>	<i>AUPRC</i>	<i>Avg latency (ms)</i>
A	1.0000	0.0000	1.0000	1.0000	1.0000	1.0000	44,707.61
B ($\tau=0.60$)	0.0000	1.0000	1.0000	0.0000	0.7500	0.8029	46,051.11
B ($\tau=0.45$)	0.8000	0.2000	1.0000	0.0000	0.7500	0.8029	52,567.63
C	0.8667	0.1333	0.9667	1.0000	1.0000	1.0000	164,822.66

Method A achieved perfect performance on the evaluated subset: it produced valid JSON for all inputs, made no errors and did not defer on any example. While Method A performed extremely strongly on the controlled subset and produced perfectly valid structured outputs, these results should be interpreted with caution. The evaluation set was relatively small and curated, which may make the task easier than a larger, more diverse benchmark subset. Therefore, the strong performance of direct generation should not be over-generalized without broader testing.

Method B behaved very differently: with the default threshold ($\tau = 0.60$) it deferred on all examples, giving zero coverage. Lowering the threshold to $\tau = 0.45$ significantly changed the behavior of Method B: coverage increased from 0.00 to 0.80, and the model stopped deferring on all examples. However, this improvement in coverage did not translate into meaningful predictive performance. The model effectively predicted NON_TOXIC for all covered toxic examples: $tp = 0$, $fn = 8$, $FNR = 1.0$

These results show that although threshold calibration can adjust abstention behavior, it does not fix the underlying weakness of Method B as a hard decision rule on this subset. The score distribution contains some ranking signal (as reflected by non – random AUROC and AUPRC values), but the final classification policy remains ineffective. This highlights a key insight: calibration alone is insufficient when the underlying scoring mechanism lacks discriminative power for the task.

Method C achieved a coverage of 0.8667, deferring on 13.3% of the inputs. JSON validity remained high (0.9667) and latency was substantially larger than the other methods due to the multiple stochastic samples required per decision (~165 ms on average). For the examples on which Method C did issue a decision, performance was perfect: precision, recall and F1 were all 1.0: $tn = 14$, $fp = 0$, $fn = 0$, $tp = 12$. This indicates that self – consistency mechanism was highly reliable on covered examples, while using abstention appropriately for uncertain cases. These results position Method C as a strong uncertainty – aware decision mechanism: it achieves excellent predictive quality with a modest defer rate, but at a significantly higher computational cost.

Combined with the earlier findings, the comparative picture becomes clear:

- . Method A: perfect performance on this subset, no abstentions and lowest latency
- . Method B: retains some ranking signal but remains weak as a hard decision rule even after threshold calibration
- . Method C: highly accurate with meaningful uncertainty handling, but the slowest due to multiple sampling rounds

This aligns precisely with the assignment’s objective to compare A/B/C in terms of accuracy, DEFER behavior, and computational cost.

<i>Method</i>	<i>Result</i>	<i>Valid JSON</i>	<i>Comment</i>
A Direct	TOXIC	yes	Correct and the fastest
B Log-likelihood	DEFER	yes	conservative / uncertain
C Self-consistency	TOXIC	yes	strongest agreement, slowest

Dataset check:

. Total examples	154
. Toxic	73
. Non – Toxic	81
. Columns	text, label
. Label Format	1 toxic / 0 non – toxic

5. Safety, DEFER and Threshold Behavior

DEFER was implemented as the explicit abstention mechanism required by the assignment. In Method A, confidence never fell below the selected threshold on the evaluated subset, resulting in full coverage (1.00). In Method C, a small number of examples were deferred due to insufficient vote agreement or low mean confidence, which slightly reduced coverage but preserved perfect predictive quality on all covered examples. Method B provided the clearest illustration of threshold – driven abstention: at $\tau = 0.60$ it deferred on every example, while at $\tau = 0.45$ it produced more decisions but still failed to correctly identify toxic positives. These behaviors highlight why coverage – risk trade – offs and careful operating point selection are essential components of uncertainty – aware moderation systems.

6. Robustness and Mitigation

Robustness experiments run with Method A on a small 8 – example subset because robustness testing is computationally expensive: each example must be evaluated in clean, attacked and mitigated forms. Five attack types were used: typo/noise, character repetition, spacing attack, leetspeak, benign quoted toxic content. A normalization function was used as mitigation.

<i>Attack</i>	<i>F1 attacked</i>	<i>F1 mitigated</i>
Clean	1.000	1.000
Character repetition	1.000	1.000
Spacing attack	1.000	1.000
Typos/noise	1.000	1.000
Leetspeak	0.909	1.000
Benign quote	0.000	0.000

System was highly robust to most surface – level perturbations. Typo/noise, spacing attacks and repeated characters did not reduce performance. Leetspeak caused a small drop in F1 from 1.00 to 0.909, but the normalization – based mitigation restored performance to 1.00. In contrast, benign quoted toxic language caused a complete failure before and after mitigation. This is an important finding: simple normalization is useful for surface obfuscation, but it

does not resolve deeper contextual ambiguity such as distinguishing direct abuse from quoted or reported harmful language.

7. Consistency Analysis

Consistency analysis was evaluated for Method C by repeating predictions across 3 different random seeds (11, 22, 33). On a 10 – example subset, the mean agreement across seeds was 0.9333 with a standard deviation of 0.1333. This indicates that self – consistency mechanism is relatively stable despite its stochastic sampling procedure. Although Method C introduces randomness through multiple sampled judgments, its final majority – vote decisions remained highly consistent across repeated runs.

Methods A and B are deterministic under fixed decoding and threshold settings, so their consistency is trivially 1.0. Therefore, Method C is the only mechanism requiring explicit consistency evaluation, and the results confirm that its stochasticity does not lead to unstable or unreliable predictions.

8. Discussion

The experiments highlight clear differences between the three decision mechanisms. Method A provided the strongest practical performance on the evaluated subset: it was fast, always produced valid JSON and achieved perfect predictive metrics on all examples. However, this result should be interpreted with caution because the evaluation subset was relatively small and controlled, which may make the task easier than a larger benchmark.

Method C matched the predictive quality of Method A on covered examples while offering a meaningful uncertainty signal through vote agreement. Its defer behavior is well – calibrated, abstaining only when agreement was low. The main drawback was computational cost: because Method C requires multiple stochastic generations, it was by far the slowest mechanism. This illustrates a key trade – off in LLM-as-a-Judge systems: stronger uncertainty modeling often comes at the expense of latency.

Method B behaved very differently. Although its AUROC and AUPRC values indicated that the log – likelihood scores contained some ranking signal, its hard predictions were weak. Under the default threshold ($\tau = 0.60$), it deferred on all examples; lowering the threshold increased coverage but did not improve predictive accuracy. This demonstrates that threshold

calibration alone cannot compensate for a weak decision rule and highlights the importance of evaluating both coverage and risk when designing abstention policies.

Robustness experiments revealed that the system is resilient to surface – level perturbations such as typos, spacing attacks, repeated characters and leetspeak variants. However, benign quoted toxic language remained a major failure mode even after normalization. This limitation reflects a deeper challenge: distinguishing between harmful intent and quoted or reported toxic content requires contextual reasoning that small generative models cannot reliably perform in a single – turn setting.

Consistency analysis showed that Method C remained stable across different random seeds, with high agreement despite its stochastic sampling process. Methods A, B are deterministic under fixed decoding settings, so their consistency is trivially perfect. System demonstrates reliable behavior across seeds, but contextual ambiguity remains a significant limitation.

Finally, ethical trade – offs are clear. Over – blocking benign content can suppress legitimate expression and under – blocking harmful content can expose users to toxicity. DEFER serves as a safety mechanism that reduces risk by abstaining on uncertain cases, but it also increases moderation workload. Balancing coverage, risk, latency, fairness remains a central challenge for LLM – based moderation systems.

9. Ethics and Limitations

Toxicity moderation involves several trade – offs. A false positive may incorrectly suppress benign user expression, while a false negative may allow genuinely harmful content to remain visible. These risks highlight the tension between over / under – blocking, a central challenge in automated moderation. DEFER served as a safety mechanism by allowing the system to abstain on uncertain cases rather than forcing a potentially harmful decision. However, abstention increases moderation workload and may shift responsibility to human reviewers, raising questions about scalability and operational fairness.

Fairness and unintended bias are also important considerations. Although the evaluation subset used, was small and controlled, real – world datasets often contain demographic or linguistic imbalances that can lead to disparate error rates across groups. Because the model was not fine – tuned and relies solely on prompting, any biases present in the underlying LLM or dataset may propagate into moderation decisions. A more comprehensive fairness analysis would require larger and more diverse evaluation sets.

Several limitations arise from the use of a small generative model operating in a single – turn setting. System struggles with contextual ambiguity, such as distinguishing between harmful intent and quoted toxic language. Sarcasm, indirect insults and multi – turn conversational dependencies remain challenging for the model. These limitations reflect the inherent constraints of small LLMs without fine – tuning and highlight the need for richer context modeling or multi – stage pipelines in real deployments.

Finally, computational constraints played a significant role. The assignment required modest hardware and 4-bit quantization, which limited throughput and increased latency; especially for Method C, which requires multiple stochastic generations. While this setup is realistic for lightweight moderation systems, it restricts the complexity of the models and techniques that can be used. Overall, the system demonstrates strong performance on a controlled subset, but broader evaluation and advanced mitigation strategies would be necessary for deployment – grade reliability.

10. Conclusion

This project implemented a complete LLM-as-a-Judge toxicity moderation pipeline using *Gemma-2-2b-it* model, strict JSON outputs constraints and uncertainty – aware abstention. Three decision mechanisms were developed / compared under identical conditions. Method A (direct generation) provided the strongest practical baseline on the evaluated subset, achieving perfect predictive performance with full coverage and low latency. Method C (self – consistency) delivered similarly strong accuracy, but at a substantially higher computational cost due to multiple sampled judgments. Method B (log – likelihood scoring) retained some ranking ability but was weak to hard decisions even after threshold calibration, demonstrating that confidence – based abstention alone can’t compensate for limited discriminative power.

Robustness experiments showed that the system is resilient to surface – level obfuscation attacks such as typos, spacing manipulation, repeated characters and most leetspeak variants. However, benign quoted toxic language remained a persistent failure mode, highlighting the limitations of small generative models in distinguishing harmful intent from contextual or reported content. Consistency analysis confirmed that Method C remained stable across different random seeds, while Methods A and B were deterministic under fixed settings.

Overall, the results show that a small generative LLM can function effectively as a structured moderation judge without fine – tuning, provided that strict schema enforcement, abstention policies and robustness evaluation are treated as core design components.

References

- [1] Borkan, D. et al. (2019). Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. WWW.
- [2] Ebrahimi, J. et al. (2018). HotFlip: White-Box Adversarial Examples for Text Classification. ACL.
- [3] Guo, C. et al. (2017). On Calibration of Modern Neural Networks. ICML.
- [4] Jiang, Z. et al. (2021). How Can We Know When Language Models Know? On the Calibration of Language Models. NeurIPS.
- [5] Kadavath, S. et al. (2022). Language Models (Mostly) Know What They Know. arXiv.
- [6] Wang, X. et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv.